

F5 AI Security Guardrails on Amazon EKS

This document captures the end-to-end deployment of F5 AI Security Guardrails on a single-node Amazon EKS cluster for a personal proof of concept.

The deployment follows the F5 AI Security Kubernetes Helm path and uses:

- Amazon EKS
- One GPU managed node group
- F5 AI Security Operator installed from Harbor
- Guardrails inference enabled
- Red Team disabled
- In-cluster PostgreSQL
- NGINX Ingress Controller
- Route 53 DNS
- Self-signed TLS certificate for lab access

This is a PoC build, not a production reference architecture. Secrets, license values, passwords, and registry credentials are intentionally excluded.

Final Environment

Item	Value
Public URL	https://guardrails.f5demo.io
AWS region	us-east-1
EKS cluster	f5-guardrails-poc
Kubernetes version	1.35
Managed node group	guardrails-gpu-ng
Instance type	g5.4xlarge
GPU	1 NVIDIA GPU
Node disk	150 GiB gp3
Database	In-cluster PostgreSQL
F5 operator namespace	f5-ai-sec
Moderator namespace	cai-moderator
Prefect namespace	prefect
Inference namespace	f5-ai-sec-inference
Ingress namespace	ingress-nginx

Architecture

```

Browser
|
| https://guardrails.f5demo.io
v
Route 53 CNAME
|
v
AWS Network Load Balancer
|
v
NGINX Ingress Controller
|-- /auth -> cai-moderator:8080 (Keycloak)
|
+-- /      -> cai-moderator:5500 (F5 AI Security UI/API)

```

Application namespaces:

```

f5-ai-sec
- F5 AI Security Operator

```

```

cai-moderator
- Moderator UI/API
- Embedded Keycloak auth
- PostgreSQL

```

```

prefect
- Prefect server
- Prefect worker
- Workflow registration job

```

```

f5-ai-sec-inference
- F5 inference service
- Guardrails scanner model

```

Important Sizing Note

The F5 runbook recommends more headroom for single-node deployments. For this PoC, `g5.4xlarge` was selected as the smallest practical AWS instance because it provides:

- 1 x NVIDIA A10G GPU with 24 GiB VRAM
- 16 vCPU
- 64 GiB memory

Observed Kubernetes allocatable resources were approximately:

```

nvidia.com/gpu: 1
cpu:            15890m
memory:         62117664Ki

```

If pods remain Pending, are OOMKilled, or model loading becomes unstable, resize to `g5.8xlarge`.

Prerequisites

Local tools:

```

aws --version
kubectl version --client=true
helm version --short
eksctl version

```

The automation script checks these tools before deployment and reports missing prerequisites in a readable checklist.

Required inputs:

- F5 AI Security license string
- Harbor registry username and password for `harbor.calypsoai.app`

- AWS CLI installed and configured with credentials that can create EKS, EC2, IAM, CloudFormation, ELB, EBS, and Route 53 resources
- Public hosted zone for the selected hostname
- LLM provider credentials for post-install application configuration

AWS credentials are not stored in this repository. The script uses the normal AWS CLI credential chain, so each machine that runs it must have one of these configured before deployment:

```
aws configure
# or
export AWS_PROFILE=<profile-name>
aws sso login --profile <profile-name>
```

Validate access before deploying:

```
aws sts get-caller-identity
```

The PoC used:

```
Hostname: guardrails.f5demo.io
Database: in-cluster PostgreSQL
Product: Guardrails only
Red Team: disabled
TLS:      self-signed certificate
```

Preflight Checks

Run the built-in prerequisite check before creating AWS resources:

```
./scripts/guardrails-poc.sh preflight
```

The preflight check validates:

- Required local tools: `aws`, `kubectl`, `helm`, `eksctl`, `openssl`, `python3`, and `curl`
- AWS credentials and selected region access
- Route 53 hosted zone for the configured hostname
- GPU instance type availability in the selected region
- EC2 GPU On-Demand quota visibility
- Harbor credentials file or environment variables
- Harbor registry login
- F5 license file or environment variable
- `eksctl` cluster configuration dry-run

If anything is missing, fix the item marked `[FAIL]` and rerun `preflight`.

Verify AWS identity:

```
aws sts get-caller-identity
```

Confirm default region:

```
aws configure get region
```

List existing EKS clusters:

```
aws eks list-clusters --region us-east-1
```

Check GPU instance availability:

```
aws ec2 describe-instance-type-offerings \
  --region us-east-1 \
  --location-type availability-zone \
```

```
--filters Name=instance-type,Values=g5.4xlarge
```

Check GPU quota:

```
aws service-quotas list-service-quotas \
  --service-code ec2 \
  --region us-east-1 \
  --query "Quotas[?contains(QuotaName, 'Running On-Demand G') || contains(QuotaName, 'Running On-Demand P')].[QuotaName,QuotaCode,Value]" \
  --output table
```

Verify Route 53 hosted zone:

```
aws route53 list-hosted-zones-by-name \
  --dns-name f5demo.io \
  --output json
```

Update eksctl

The installed `eksctl` version originally supported only Kubernetes versions up to `1.31`. AWS listed `1.35` as the default supported EKS version in `us-east-1`, so `eksctl` was upgraded before cluster creation.

```
brew upgrade eksctl
eksctl version
```

Create the EKS Cluster

Dry-run the cluster configuration first:

```
eksctl create cluster \
  --name f5-guardrails-poc \
  --region us-east-1 \
  --version 1.35 \
  --managed \
  --nodegroup-name guardrails-gpu-ng \
  --node-type g5.4xlarge \
  --nodes 1 \
  --nodes-min 1 \
  --nodes-max 1 \
  --node-volume-size 150 \
  --node-ami-family AmazonLinux2023 \
  --dry-run
```

Create the cluster:

```
eksctl create cluster \
  --name f5-guardrails-poc \
  --region us-east-1 \
  --version 1.35 \
  --managed \
  --nodegroup-name guardrails-gpu-ng \
  --node-type g5.4xlarge \
  --nodes 1 \
  --nodes-min 1 \
  --nodes-max 1 \
  --node-volume-size 150 \
  --node-ami-family AmazonLinux2023 \
  --install-nvidia-plugin \
  --vpc-nat-mode Disable
```

Notes:

- NAT gateway was disabled to reduce recurring PoC cost.
- The node uses public subnet egress.
- `eksctl` installed the NVIDIA device plugin automatically because the node group used an EKS optimized accelerated AMI.

Verify the node:

```
kubectl get nodes -o wide
kubectl get node -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.allocatable.
nvidia\.com/gpu}{"\t"}{.status.allocatable.cpu}{"\t"}{.status.allocatable.memory}{"\n"}}{end}'
kubectl get pods -n kube-system -o wide
```

Expected result:

- One Ready node
- `nvidia-device-plugin-daemonset` Running
- `nvidia.com/gpu` allocatable value is 1

Prepare Harbor Pull Secret

Create the F5 namespace:

```
kubectl create namespace f5-ai-sec
```

Create the Harbor pull secret:

```
kubectl create secret docker-registry regcred \
  --docker-server='harbor.calypsoai.app' \
  --docker-username='<HARBOR_USERNAME>' \
  --docker-password='<HARBOR_PASSWORD>' \
  -n f5-ai-sec
```

Log in with Helm:

```
helm registry login harbor.calypsoai.app
```

Install the F5 AI Security Operator

```
helm upgrade --install f5-ai-security-operator \
  oci://harbor.calypsoai.app/calypsoai/f5-ai-security-operator-helm \
  --version 1.4.1 \
  -n f5-ai-sec
```

Verify:

```
helm list -n f5-ai-sec
kubectl get crd securityoperators.ai.security.f5.com
kubectl get pods -n f5-ai-sec -o wide
```

Expected:

```
controller-manager 1/1 Running
```

Apply Guardrails SecurityOperator

Create a Guardrails-only `SecurityOperator` custom resource.

The actual deployment used a generated PostgreSQL password and the F5 license from a local file. Do not commit the real license or generated passwords.

```
apiVersion: ai.security.f5.com/v1alpha1
kind: SecurityOperator
metadata:
  name: security-operator-demo
  namespace: f5-ai-sec
spec:
  registryAuth:
    existingSecret: "regcred"

  postgresql:
    enabled: true
    values:
      postgresql:
        auth:
          password: "<GENERATED_POSTGRES_PASSWORD>"
```

```

jobManager:
  enabled: true

moderator:
  enabled: true
  values:
    env:
      CAI_MODERATOR_BASE_URL: https://guardrails.f5demo.io
    secrets:
      CAI_MODERATOR_DB_ADMIN_PASSWORD: "<GENERATED_POSTGRES_PASSWORD>"
      CAI_MODERATOR_DEFAULT_LICENSE: "<F5_LICENSE_STRING>"

inference:
  enabled: true
  values:
    inference:
      guardrails:
        enabled: true
      redteam:
        enabled: false

```

Apply it:

```
kubectl apply -f securityoperator.yaml
```

Verify namespace and pod creation:

```

kubectl get securityoperator -n f5-ai-sec
kubectl get ns
kubectl get pods -n f5-ai-sec
kubectl get pods -n cai-moderator
kubectl get pods -n prefect
kubectl get pods -n f5-ai-sec-inference

```

Storage Fix: Install EBS CSI

PostgreSQL initially stayed Pending because the PVC required `ebs.csi.aws.com`, but the EBS CSI driver was not installed.

The `gp2` StorageClass also was not marked as default.

Patch `gp2` as default:

```
kubectl patch storageclass gp2 \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

Install the EKS Pod Identity Agent:

```

eksctl create addon \
  --cluster f5-guardrails-poc \
  --region us-east-1 \
  --name eks-pod-identity-agent \
  --force

```

Install the EBS CSI driver with the recommended pod identity association:

```

eksctl create addon \
  --cluster f5-guardrails-poc \
  --region us-east-1 \
  --name aws-ebs-csi-driver \
  --auto-apply-pod-identity-associations \
  --force

```

Verify:

```

aws eks describe-addon \
  --cluster-name f5-guardrails-poc \
  --addon-name aws-ebs-csi-driver \
  --region us-east-1 \
  --query 'addon.[status,health.issues]' \
  --output json

```

```
kubectl get pods -n kube-system \
```

```
-l app.kubernetes.io/name=aws-ebs-csi-driver \
-o wide
```

```
kubectl get pvc -n cai-moderator -o wide
```

Expected:

- EBS CSI add-on **ACTIVE**
- EBS CSI controller and node pods Running
- PostgreSQL PVC Bound

Prefect Workflow Registration Fix

The first workflow registration jobs failed because they started before the Prefect API was ready.

Verify Prefect API health from inside the cluster:

```
kubectl exec -n prefect deploy/prefect-server -- \
/bin/sh -c 'python - <<PY
import urllib.request
print(urllib.request.urlopen("http://prefect-server.prefect.svc.cluster.local:4200/api/health",
timeout=5).read().decode()[:200])
PY'
```

Expected:

```
true
```

Delete failed early jobs:

```
kubectl delete job -n prefect \
  cai-workflows-1781048696 \
  cai-workflows-1781048699 \
  --ignore-not-found
```

Create a fresh workflow registration job:

```
kubectl create job -n prefect \
  cai-workflows-manual-20260610-0054 \
  --from=cronjob/cai-workflows
```

Verify:

```
kubectl get pods -n prefect -o wide
kubectl logs -n prefect job/cai-workflows-manual-20260610-0054 --tail=100
```

Expected:

- Workflow registration job Completed
- Prefect server Running
- Prefect worker Running

Install NGINX Ingress Controller

Add and update the Helm repository:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update ingress-nginx
```

Install ingress-nginx:

```
helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  --set controller.service.annotations.service.beta.kubernetes.io/aws-load-balancer-type=nlb \
  --set controller.service.annotations.service.beta.kubernetes.io/aws-load-balancer-scheme=internet-facing \
```

```
--set controller.service.externalTrafficPolicy=Local
```

Verify the AWS load balancer hostname:

```
kubectl get svc -n ingress-nginx ingress-nginx-controller -o wide
```

Create TLS Secret

For this PoC, a self-signed certificate was used.

```
tmpdir=$(mktemp -d)
```

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout "$tmpdir/tls.key" \
  -out "$tmpdir/tls.crt" \
  -subj "/CN=guardrails.f5demo.io" \
  -addext "subjectAltName=DNS:guardrails.f5demo.io"
```

```
kubectl create secret tls cai-moderator-tls \
  -n cai-moderator \
  --cert="$tmpdir/tls.crt" \
  --key="$tmpdir/tls.key" \
  --dry-run=client \
  -o yaml | kubectl apply -f -
```

```
rm -rf "$tmpdir"
```

Browser warning is expected because the certificate is self-signed.

Create Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: cai-moderator-ingress
  namespace: cai-moderator
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-http-version: "1.1"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - guardrails.f5demo.io
    secretName: cai-moderator-tls
  rules:
  - host: guardrails.f5demo.io
    http:
      paths:
      - path: /auth
        pathType: Prefix
        backend:
          service:
            name: cai-moderator
            port:
              number: 8080
      - path: /
        pathType: Prefix
        backend:
          service:
            name: cai-moderator
            port:
              number: 5500
```

Apply:

```
kubectl apply -f cai-moderator-ingress.yaml
```

Verify:


```
kubectl get ingress -n cai-moderator -o wide
```

Create Route 53 Record

Create a CNAME from the public hostname to the ingress-nginx load balancer hostname.

```
aws route53 change-resource-record-sets \
  --hosted-zone-id <F5DEMO_IO_HOSTED_ZONE_ID> \
  --change-batch file://guardrails-cname.json
```

Example change batch:

```
{
  "Comment": "Point Guardrails PoC hostname to ingress-nginx load balancer",
  "Changes": [
    {
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "guardrails.f5demo.io.",
        "Type": "CNAME",
        "TTL": 60,
        "ResourceRecords": [
          {
            "Value": "<INGRESS_NGINX_LOAD_BALANCER_HOSTNAME>"
          }
        ]
      }
    ]
  }
}
```

Verify the change:

```
aws route53 get-change --id <CHANGE_ID>
```

Expected:

INSYNC

Final Validation

Check the public UI route:

```
curl -k -I --max-time 20 https://guardrails.f5demo.io/
```

Expected:

HTTP/2 200

Check the auth route:

```
curl -k -I --max-time 20 https://guardrails.f5demo.io/auth/
```

Expected:

HTTP/2 302

location: https://guardrails.f5demo.io/auth/admin/

Final pod check:

```
kubectl get pods -n f5-ai-sec -o wide
kubectl get pods -n cai-moderator -o wide
kubectl get pods -n prefect -o wide
kubectl get pods -n f5-ai-sec-inference -o wide
```

Expected state:

```
f5-ai-sec:
  controller-manager          1/1 Running

cai-moderator:
  cai-moderator              1/1 Running
  cai-moderator-postgres-cai-postgresql-0  1/1 Running
```

```

prefect:
  prefect-server          1/1 Running
  prefect-worker          1/1 Running
  cai-workflows-manual-*  Completed

f5-ai-sec-inference:
  f5-ai-sec-inference    1/1 Running
  model-cai-phi-4-gptq-4bit 1/1 Running

```

First Login and Password Change

Initial credentials from the F5 runbook:

```

Username: admin
Password: pass

```

The main Guardrails UI account page does not expose password management.

Use the embedded Keycloak account console. For this PoC deployment, the Guardrails UI login uses the **master** realm:

```
https://guardrails.f5demo.io/auth/realms/master/account/#/security/signingin
```

The deployment can also expose a **calypsoai** realm:

```
https://guardrails.f5demo.io/auth/realms/calypsoai/account/#/security/signingin
```

Keycloak passwords are realm-specific. Changing the **admin** password in **master** does not automatically change the **admin** password in **calypsoai**. If both realms accept the initial password, rotate the password in both account consoles.

After login, change the password under **Account Security** -> **Signing in** -> **Update password**.

Useful Commands

Cluster:

```

kubectl get nodes -o wide
kubectl top nodes
kubectl get pods -A

```

F5 namespaces:

```

for ns in f5-ai-sec cai-moderator prefect f5-ai-sec-inference ingress-nginx; do
  echo "=== $ns ==="
  kubectl get pods -n "$ns"
done

```

Moderator logs:

```
kubectl logs -n cai-moderator deploy/cai-moderator --tail=100
```

Operator logs:

```
kubectl logs -n f5-ai-sec deploy/controller-manager --tail=100
```

Inference logs:

```
kubectl logs -n f5-ai-sec-inference deploy/f5-ai-sec-inference --tail=100
```

Prefect logs:

```

kubectl logs -n prefect deploy/prefect-server --tail=100
kubectl logs -n prefect deploy/prefect-worker --tail=100

```

Storage:

```

kubectl get pvc -n cai-moderator
kubectl describe pvc -n cai-moderator

```

Ingress:

```
kubectl get svc -n ingress-nginx ingress-nginx-controller -o wide
kubectl get ingress -n cai-moderator -o wide
```

Cleanup

To remove the PoC cluster and the AWS resources created by `eksctl`:

```
eksctl delete cluster \
  --name f5-guardrails-poc \
  --region us-east-1
```

Then remove the Route 53 CNAME if it is no longer needed.

Cost-Control Automation

The repository includes a reusable automation script for deleting the PoC when it is not needed and recreating it later with minimal interaction.

```
./scripts/guardrails-poc.sh preflight
./scripts/guardrails-poc.sh up
./scripts/guardrails-poc.sh status
./scripts/guardrails-poc.sh down --yes
```

The script covers the full deployment path used in this PoC, including the issues encountered during the first manual deployment:

- EKS cluster creation with a single GPU node
- NVIDIA device plugin installation through `eksctl`
- EKS Pod Identity Agent installation
- AWS EBS CSI driver installation
- `gp2` StorageClass default annotation patch
- Harbor pull secret creation
- F5 AI Security Operator installation
- Guardrails-only `SecurityOperator` creation
- In-cluster PostgreSQL configuration
- Prefect workflow registration retry after Prefect API readiness
- NGINX Ingress Controller installation
- Self-signed TLS secret creation
- `/auth` and `/` ingress routing
- Route 53 CNAME upsert and delete
- Post-delete cleanup of available EBS volumes still tagged as owned by the PoC cluster
- Public URL validation

Before deployment, the script runs a preflight gate and prints a readable checklist for missing tools, AWS access, Route 53, Harbor credentials, license input, GPU instance availability, and `eksctl` cluster configuration.

Configure Automation

Copy the example config:

```
cp config/guardrails-poc.env.example config/guardrails-poc.env
```

Edit `config/guardrails-poc.env` if you want to change cluster name, region, instance type, hostname, hosted zone, or secret file locations.

By default, the script references:

```
HARBOR_CREDENTIALS_FILE="config/harbor.txt"
F5_LICENSE_FILE="config/license.txt"
GUARDRAILS_DEFAULT_USERNAME="admin"
GUARDRAILS_DEFAULT_PASSWORD="pass"
GUARDRAILS_AUTH_REALMS="master calypsoai"
```

Relative paths in `config/guardrails-poc.env` are resolved from the repository root, so the same config works after cloning the repo on another machine.

The Harbor credentials file format is:

```
<harbor username>
<harbor password>
```

The license file should contain the license string as its full content.

These local secret files are ignored by Git. Create them after cloning:

```
printf '%s\n%s\n' '<harbor username>' '<harbor password>' > config/harbor.txt
printf '%s\n' '<F5 license string>' > config/license.txt
chmod 600 config/harbor.txt config/license.txt
```

You can also override secrets at runtime without changing the config file:

```
export HARBOR_USERNAME='<new-harbor-username>'
export HARBOR_PASSWORD='<new-harbor-password>'
export F5_LICENSE_STRING='<new-license-string>'
```

```
./scripts/guardrails-poc.sh up
```

The script stores the generated PostgreSQL password in:

```
.secrets/postgres-password
```

That directory is ignored by Git. Remove the file if you want a fresh generated PostgreSQL password on the next deployment.

AWS Credentials on Another Machine

The script does not store or manage AWS credentials. It assumes the AWS CLI is already installed and authenticated on the machine running the script.

On a new machine, configure AWS before running `up`, `down`, or `status`:

```
aws configure
aws sts get-caller-identity
aws configure get region
```

Use an IAM user or role with permission for:

- EKS cluster and add-on lifecycle
- EC2 instances, VPCs, security groups, EBS volumes, and network interfaces
- Elastic Load Balancing
- IAM roles and policies created by `eksctl`
- CloudFormation stacks created by `eksctl`
- Route 53 hosted zone record changes for the configured hostname

If you use AWS SSO or named profiles, authenticate first and pass the profile through the environment:

```
aws sso login --profile <profile-name>
AWS_PROFILE=<profile-name> ./scripts/guardrails-poc.sh up
```

Test Without Touching AWS

Use dry-run mode:

```
./scripts/guardrails-poc.sh --dry-run up
./scripts/guardrails-poc.sh --dry-run status
./scripts/guardrails-poc.sh --dry-run down --yes
```

Start the PoC

Check prerequisites first:

```
./scripts/guardrails-poc.sh preflight
```

Then deploy:

```
./scripts/guardrails-poc.sh up
```

Expected runtime is roughly 25-45 minutes because the script creates a new EKS control plane, GPU node, add-ons, images, model pod, ingress load balancer, and DNS record.

At the end, the script prints a concise status summary with the UI URL, initial login credentials, and Keycloak account-console links for changing the password.

Stop the PoC and Minimize Cost

```
./scripts/guardrails-poc.sh down --yes
```

This deletes:

- Route 53 `guardrails.f5demo.io` CNAME
- EKS cluster
- GPU node group
- AWS load balancer
- EBS volumes created through the cluster
- Any available leftover EBS volumes still tagged `kubernetes.io/cluster/f5-guardrails-poc=owned`
- CloudFormation stacks created by `eksctl`

This delete-and-recreate model is slower than scaling a node group to zero, but it removes the EKS control-plane and GPU-node cost instead of merely reducing compute cost.

After teardown, `status` should show a clean stopped state:

```
Guardrails PoC status
=====
Environment:      stopped
EKS cluster:      not found (f5-guardrails-poc in us-east-1)
DNS record:       deleted (guardrails.f5demo.io not present)
EBS volumes:      none found with cluster-owned tag
Kubernetes resources: skipped because the EKS cluster is deleted
Public URL:       offline (https://guardrails.f5demo.io)

Next actions:
  Start again:     ./scripts/guardrails-poc.sh up
  Re-check status: ./scripts/guardrails-poc.sh status
```

Production Considerations

For production, change the following:

- Use a trusted TLS certificate from ACM or cert-manager.
- Use external PostgreSQL such as Amazon RDS.

- Use private node networking and controlled egress.
- Enable backup and restore for PostgreSQL.
- Use larger or separate node groups for CPU and GPU workloads.
- Use `g5.8xlarge` or larger for single-node stability.
- Avoid embedding secrets in shell history or static manifests.
- Store credentials in AWS Secrets Manager, External Secrets Operator, Sealed Secrets, or another approved secret manager.
- Configure monitoring, logging, and alerting.
- Validate license entitlements before enabling Red Team.

References

- F5 AI Security documentation: <https://docs.aisecurity.f5.com/>
- F5 AI Security Helm install guide: <https://docs.aisecurity.f5.com/get-started/get-started-helm.html>
- F5 AI Security API docs: <https://docs.aisecurity.f5.com/api-docs/>
- Amazon EKS documentation: <https://docs.aws.amazon.com/eks/>
- ingress-nginx Helm chart: <https://github.com/kubernetes/ingress-nginx>